

- Mareana — Cloud & DevOps Architect
- L2 Final Round (Director-Level) Interview Preparation
 - What Directors Test in Final Rounds
 - Table of Contents
 - 1. Live Whiteboard Architecture Scenarios
 - Scenario 1: "Design Mareana's Cloud Platform from Scratch — You have 3 months and a 5-person team."
 - Scenario 2: "A major manufacturing client says their data is processed 4 hours after upload. They need it in under 10 minutes. What do you do?"
 - Scenario 3: "We want zero-downtime deployments but our EKS pods take 60 seconds to start up. How do you solve this?"
 - 2. Production Debugging Walkthroughs
 - Debug Scenario 1: "Our ML inference endpoint is responding in 800ms. SLA is 200ms. 3 PM today. Production. Go."
 - Debug Scenario 2: "Pods are randomly crashing every 2-3 hours. No pattern. No error in logs. What do you do?"
 - Debug Scenario 3: "Terraform apply succeeded but EKS nodes are not joining the cluster. Describe your debug process."
 - 3. Customer Delight Stories
 - Story 1: Proactive Anomaly — Saving a Customer Before They Know
 - Story 2: Reducing Time-to-Value for a New Customer Onboarding
 - Story 3: 3 AM Production Incident — Customer Was Watching
 - 4. Trade-off & Decision-Making Questions
 - Q: "SageMaker vs. Self-hosted ML inference on EKS — how do you decide?"
 - Q: "ArgoCD vs. FluxCD — which GitOps tool would you pick for Mareana and why?"
 - Q: "A developer wants to deploy directly to production via kubectl. How do you handle it?"
 - Q: "We're over budget by 30% on AWS. Engineering leadership wants you to cut costs in 2 weeks. What's your plan?"
 - 5. Platform Strategy & Roadmap Thinking
 - Q: "How would you build a Platform Engineering practice at Mareana from scratch?"
 - Q: "What's the biggest platform mistake you see companies make?"
 - 6. Failure & Learning Questions

- Q: "Tell me about the biggest production incident you caused or that happened on your watch."
- Q: "Tell me about a technology decision you made that you later regretted."
- 7. Leadership, Influence & Cross-Functional
 - Q: "How do you get a security team to approve a Kubernetes architecture when they don't understand containers?"
 - Q: "How do you handle a product manager who wants to skip the staging environment to ship faster?"
 - Q: "How do you influence technical decisions when you don't have direct authority — e.g., a product team that reports to a different VP?"
- 8. Mareana Domain Deep-Dives
 - Q: "How would you architect the data pipeline for a manufacturing plant producing 10,000 sensor readings per second?"
 - Q: "Mareana's AI model predicts equipment failure. How do you ensure that model keeps working accurately in production 12 months from now?"
- 9. Director's Rapid-Fire Round
- 10. Questions YOU Should Ask the Director
- Final Prep Checklist Before Your F2F
- The One Thing to Remember

Mareana — Cloud & DevOps Architect

L2 Final Round (Director-Level) Interview Preparation

Round: Final Face-to-Face | **Interviewer:** Director of Engineering / VP Engineering

Format: 60–90 minutes | Deep-dive technical + Approach + Customer Delight + Leadership **Build on:** L1 document (concepts already established) **Focus:** HOW you think → HOW you solve → HOW you delight customers → HOW you lead

What Directors Test in Final Rounds

What They Probe	What They Really Want to See
Solution Approach	Do you structure problems or jump to solutions?
Debugging under pressure	Can you diagnose live production issues calmly?
Trade-off thinking	Do you understand cost vs. speed vs. risk?
Customer delight	Do you think beyond SLAs to actual user experience?
Failure handling	Are you honest, learning-focused, and resilient?
Platform strategy	Can you think 12–24 months ahead?
Leadership influence	Can you drive decisions without direct authority?
Mareana domain fit	Do you understand Manufacturing AI at scale?

Table of Contents

1. [Live Whiteboard Architecture Scenarios](#)
2. [Production Debugging Walkthroughs](#)
3. [Customer Delight Stories](#)
4. [Trade-off & Decision-Making Questions](#)
5. [Platform Strategy & Roadmap Thinking](#)
6. [Failure & Learning Questions](#)
7. [Leadership, Influence & Cross-Functional](#)
8. [Mareana Domain Deep-Dives](#)
9. [Director's Rapid-Fire Round](#)
10. [Questions YOU Should Ask the Director](#)

1. Live Whiteboard Architecture Scenarios

These are open-ended — the director gives you a problem and watches HOW you break it down. **Always think out loud.**

Scenario 1: "Design Mareana's Cloud Platform from Scratch — You have 3 months and a 5-person team."

How to approach (say this out loud):

```
"Before I draw anything, let me understand the constraints:
- How many tenants (customers) are we targeting at launch?
- Is this SaaS (shared) or dedicated per customer?
- What's the primary data source — IoT sensors, ERP, manual uploads?
- What's the SLA requirement — 99.9%? 99.99%?
- Any regulatory constraints — data residency, GDPR, ISO 27001?
- Budget ballpark?"
```

Then structure your answer in phases:

Month 1 — Foundation (Week 1–4)

Week 1–2: Landing Zone

- └─ AWS Organizations + Control Tower setup
- └─ 3 accounts: prod, staging, shared-services
- └─ Terraform baseline (networking, IAM, KMS, CloudTrail)
- └─ GitHub repo + branch protection
- └─ Slack + PagerDuty integration

Week 3–4: Core Infrastructure

- └─ VPC (3 AZs, private subnets, VPC endpoints)
- └─ EKS cluster (private, 1.29, managed node groups)
- └─ ECR + basic CI pipeline (GitHub Actions → ECR)
- └─ Aurora PostgreSQL (multi-AZ) + ElastiCache Redis
- └─ ArgoCD deployed — GitOps from day 1

Month 2 — Platform Services (Week 5–8)

Week 5–6: Data Ingestion Layer

- └─ Kinesis Data Streams (IoT telemetry)
- └─ S3 Data Lake (Bronze/Silver/Gold)
- └─ AWS Glue (ETL for data processing)
- └─ API Gateway + Lambda (REST ingestion)

Week 7–8: Observability + Security

- └─ kube-prometheus-stack (metrics + alerting)
- └─ Fluent Bit → CloudWatch Logs
- └─ GuardDuty + Security Hub enabled

- └ Trivy + tfsec in CI pipeline
- └ SLA dashboards for each service

Month 3 — Multi-tenancy + ML Infrastructure (Week 9–12)

Week 9–10: Multi-tenancy

- └ Namespace-per-tenant with NetworkPolicies
- └ Karpenter (dynamic scaling)
- └ KEDA (event-driven scaling from SQS/Kinesis)
- └ Kubecost (per-tenant cost attribution)

Week 11–12: ML Inference Layer

- └ SageMaker endpoints OR custom EKS GPU node pool
- └ Feature store (ElastiCache Redis)
- └ Model serving via NVIDIA Triton on g5.xlarge
- └ DR drill + load test before go-live

Trade-offs you proactively mention:

"I'm choosing EKS over ECS because Mareana likely needs Kubernetes-native tooling for ML workloads, portability, and community ecosystem. The overhead is justified at scale. For early-stage, ECS Fargate would be faster to launch but harder to migrate later."

Scenario 2: "A major manufacturing client says their data is processed 4 hours after upload. They need it in under 10 minutes. What do you do?"

Approach (structure it as Discover → Diagnose → Design → Deliver):

Step 1: Discover — Ask before assuming

Questions to ask:

- └ What's the current pipeline? (Batch ETL? Scheduled Glue jobs? Cron?)
- └ Data volume per upload? (100MB vs 100GB changes everything)
- └ What "processed" means – transformed in S3? or available in the app UI?
- └ Is this all customers or one customer's specific pattern?
- └ Is 10 minutes a contractual SLA or a wish?

Step 2: Diagnose — Map current pipeline

Current (4 hours):

Upload → S3 → Glue job (scheduled every 4h) → Redshift → API → UI

↑

Problem: Scheduled batch – waits up to 4 hours even if file lands early

Step 3: Design — Event-driven pipeline

New (< 10 minutes):

File lands in S3

↓ (S3 Event Notification → EventBridge)

↓ (< 1 second trigger)

Lambda (validate file, queue work)

↓ (< 30 seconds)

SQS Queue → EKS KEDA worker (auto-scales on queue depth)

↓ (< 5 minutes – parallel processing)

Glue Streaming / Spark on EKS

↓

S3 Silver Layer → Aurora (structured data)

↓ (< 30 seconds)

EventBridge → WebSocket notification → Client UI

↓

Customer sees "Processing Complete" in UI

Total: < 8 minutes

Step 4: Deliver — Rollout plan

Week 1: Implement S3 → EventBridge → Lambda trigger (no Glue changes)

Week 2: Replace scheduled Glue with streaming/on-demand

Week 3: Add KEDA auto-scaling for concurrent uploads

Week 4: Load test with client's actual data volume

Week 5: Canary rollout (5% of uploads) → validate → 100%

Customer delight angle (say this):

"Beyond just meeting the 10-minute SLA, I'd add a real-time progress indicator in the UI — 'File received → Validating → Processing → Ready'. Customers love visibility. A 9-minute wait feels shorter than a silent 4-minute wait."

Scenario 3: "We want zero-downtime deployments but our EKS pods take 60 seconds to start up. How do you solve this?"

Answer:

This is a multi-layer problem. Let me walk through each layer:

Layer 1: Slow container startup (60s)

Root causes to investigate:

- Slow JVM startup (Java app)?
 - Fix: Use `-XX:TieredStopAtLevel=1` (faster startup, less optimization)
 - Or: Native compilation (GraalVM native-image → 50ms startup)
 - Or: CRaC (Checkpoint/Restore in Userspace – Java 21+)
- Large Docker image (slow pull from ECR)?
 - Fix: Optimize image layers (multi-stage, cache layers)
 - ECR pull-through cache on nodes
 - Consider: containerd image pull concurrency
- Slow application initialization (DB connections, cache warm-up)?
 - Fix: Lazy initialization (Spring Boot: `spring.main.lazy-initialization=true`)
 - Connection pool pre-warming in readiness probe
- Missing pre-stop hook (pod terminates before connections drain)?
 - Fix: Add preStop sleep + `terminationGracePeriodSeconds`

Layer 2: Deployment strategy

```
# RollingUpdate with proper settings
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2          # Launch 2 new pods before terminating old
      maxUnavailable: 0    # Never reduce capacity during rollout

  template:
    spec:
      containers:
      - name: api
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
            initialDelaySeconds: 30  # Give app time to start
```

```
    periodSeconds: 5
    failureThreshold: 6          # 30 second tolerance
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 60
  periodSeconds: 10
lifecycle:
  preStop:
    exec:
      command: ["sleep", "15"]  # Allow ALB to deregister first
terminationGracePeriodSeconds: 90
```

Layer 3: ALB + Target Group Deregistration

ALB deregistration delay: Set to 30 seconds (default 300s is too long)
Target group: connection draining enabled

This ensures:

1. ALB stops sending new traffic to terminating pod
2. Pod finishes in-flight requests (15s preStop + 15s drain)
3. Then pod terminates – zero dropped connections

Layer 4: Traffic strategy for near-zero risk

Use Argo Rollouts (Canary):

- └─ 10% traffic to new version → validate 5 min
- └─ 30% → validate 5 min
- └─ 60% → validate 5 min
- └─ 100% → complete
- └─ If any 5xx spike detected → auto-rollback in 60 seconds

Customer delight angle:

"I'd also set up a maintenance window notification system — even for zero-downtime deployments, sending customers a 'We're deploying improvements' notification in the app builds trust. They see a version number change and feel progress."

2. Production Debugging Walkthroughs

Director will give you symptoms. Walk through your diagnostic process **step by step**.

Debug Scenario 1: "Our ML inference endpoint is responding in 800ms. SLA is 200ms. 3 PM today. Production. Go."

Your systematic approach:

STEP 1: ISOLATE (2 minutes)

- Is it ALL customers or ONE customer?
 - `kubectl logs -n prod -l app=inference-api --since=10m | grep -i slow`
- Is it ALL endpoints or ONE model?
 - Check CloudWatch: per-endpoint latency breakdown
- Is it sudden spike or gradual degradation?
 - CloudWatch: look at 24h graph – when did it change?
- Any recent deployments?
 - Check ArgoCD: last sync time

STEP 2: LAYER-BY-LAYER DRILL (10 minutes)

- ALB layer:
 - `aws cloudwatch get-metric-statistics \`
 - `--namespace AWS/ApplicationELB \`
 - `--metric-name TargetResponseTime \`
 - `--dimensions Name=LoadBalancer,Value=app/ml-alb/xxx`
 - Is ALB adding latency? (TargetResponseTime vs RequestCount)
- EKS/Pod layer:
 - `kubectl top pods -n prod -l app=inference-api`
 - `kubectl describe pod <pod-name> -n prod`
 - CPU throttling? Memory pressure? Node issue?
- GPU layer (ML inference):
 - `kubectl exec -it <pod> -n prod -- nvidia-smi`
 - GPU utilization: should be 70-90%
 - If <30%: model is CPU-bound or batching issue
 - If 100%: GPU bottleneck, need more replicas or batching
- Application logs:
 - `kubectl logs <pod> -n prod | grep -E "inference_time|latency"`
 - Where is time being spent: preprocessing, model inference, postprocessing?
- External dependency:
 - `kubectl exec -it <pod> -- curl -w "%{time_connect},{time_total}" \`
 - `http://feature-store-redis:6379`
 - Feature store lookup latency? (Redis should be <1ms)

STEP 3: ROOT CAUSE PATTERNS

```
|— Pattern A: GPU memory full → batching requests → queue builds up
|   Fix: Scale GPU replicas (kubectl scale --replicas=6 deploy/inference-
api)
|       Or: Reduce batch size to reduce per-request latency
|
|— Pattern B: Model not on GPU (CPU fallback)
|   Check: kubectl logs | grep "CUDA not available" or "device: cpu"
|   Fix: Verify GPU node selector + NVIDIA device plugin running
|
|— Pattern C: Feature store cold (Redis cold start)
|   Check: redis-cli INFO stats → keyspace_hits vs keyspace_misses
|   Fix: Pre-warm feature cache on pod startup
|
|— Pattern D: Model version change (slower model deployed)
|   Check: ArgoCD history → compare model SHA
|   Fix: Rollback (kubectl rollout undo deploy/inference-api)
|
|— Pattern E: Input data size increase (larger payloads)
|   Check: ALB access logs → request_size over time
|   Fix: Add input validation + payload size limit at API Gateway
```

STEP 4: FIX + VALIDATE

```
|— Apply fix with monitoring window
|— Watch CloudWatch: TargetResponseTime
|— Confirm: p50, p95, p99 all below 200ms
|— Communicate to stakeholders:
  "Root cause identified: [X]. Fix applied at 15:47. Latency back to 180ms
p99.
  Post-mortem scheduled for tomorrow 10 AM."
```

Debug Scenario 2: "Pods are randomly crashing every 2-3 hours. No pattern. No error in logs. What do you do?"

STEP 1: GET THE FACTS

```
kubectl get events -n prod --sort-by='.lastTimestamp' | tail -30
kubectl describe pod <crashed-pod> -n prod
# Look for: OOMKilled? Exit code? Last state?
```

STEP 2: EXIT CODE ANALYSIS

```
|— Exit code 137 = OOMKilled (Linux SIGKILL, killed by kernel)
|   → kubectl describe pod: "OOMKilled: true"
|   → Fix: Increase memory limit OR fix memory leak
|
|— Exit code 143 = SIGTERM (graceful shutdown requested)
|   → Karpenter consolidating nodes? Spot interruption?
|   → Check: kubectl get events | grep "Spot interruption"
|   → Fix: Add PodDisruptionBudget, use On-Demand for critical pods
```

- Exit code 1/2 = Application crash
 - Get logs from previous run:


```
kubectl logs <pod> -n prod --previous
```
- Exit code 0 = Pod exited cleanly (not a crash – bad liveness probe?)
 - Check liveness probe – is it too aggressive?

STEP 3: MEMORY LEAK INVESTIGATION (if OOMKilled)

Option A: Prometheus memory metrics

```
kubectl port-forward svc/prometheus 9090 -n monitoring
```

```
# Query: container_memory_usage_bytes{pod=~"inference-api.*",
namespace="prod"}
```

Look for: steady upward trend over 2–3 hours then drop (OOM cycle)

Option B: Heap dump analysis (Java)

```
kubectl exec -it <pod> -- jcmd 1 VM.heap_dump /tmp/heap.hprof
```

```
kubectl cp prod/<pod>:/tmp/heap.hprof ./heap.hprof
```

Analyze with Eclipse MAT or VisualVM

Option C: Memory profiling (Python/ML workload)

```
kubectl exec -it <pod> -- python -c "
```

```
import tracemalloc
```

```
tracemalloc.start()
```

```
# ... run workload ...
```

```
snapshot = tracemalloc.take_snapshot()
```

```
print(snapshot.statistics('lineno')[:10])
```

```
"
```

STEP 4: FIX OPTIONS

- Short-term: Increase memory limit (buy time)

```
resources:
```

```
  limits:
```

```
    memory: "4Gi" # was 2Gi
```

- Short-term: Set pod restart policy with alert

```
→ AlertManager: PodRestartCountHigh > 3 in 30min → PagerDuty
```

- Long-term: Fix memory leak in application code

```
→ Profile, identify leak, fix, test, deploy
```

- Long-term: Add JVM flags (Java)

```
-XX:+HeapDumpOnOutOfMemoryError
```

```
-XX:HeapDumpPath=/tmp/oom-dump.hprof
```

```
→ EFS/PVC mount at /tmp so dump survives pod death
```

Debug Scenario 3: "Terraform apply succeeded but EKS nodes are not joining the cluster. Describe your debug process."

STEP 1: CHECK NODE STATUS

```
kubectl get nodes
```

```
# Expected: Ready. Actual: Nothing shown (or NotReady)
```

```
aws ec2 describe-instances \
```

```
  --filters "Name=tag:aws:eks:cluster-name,Values=mareana-prod" \
```

```
  --query "Reservations[].Instances[." \
```

```
{ID:InstanceId,State:State.Name,LaunchTime:LaunchTime}]"
```

```
# Are instances even running?
```

STEP 2: EC2 LAUNCH ISSUES

```
# Check: Did Terraform create the ASG/Launch Template correctly?
```

```
aws autoscaling describe-auto-scaling-groups \
```

```
  --auto-scaling-group-names mareana-prod-general
```

```
# Check UserData (bootstrap script)
```

```
aws ec2 describe-launch-template-versions \
```

```
  --launch-template-id lt-xxxx --versions '$Latest' \
```

```
  --query "LaunchTemplateVersions[0].LaunchTemplateData.UserData" \
```

```
| base64 -d
```

```
# UserData must have: /etc/eks/bootstrap.sh mareana-prod
```

STEP 3: EC2 BOOT LOGS (SSM – no SSH needed)

```
aws ssm start-session --target i-0xxxx
```

```
sudo cat /var/log/cloud-init-output.log | tail -100
```

```
sudo journalctl -u kubelet -n 100
```

```
# Common errors:
```

```
# - "Unable to authenticate to the cluster" → IAM/RBAC issue
```

```
# - "Failed to connect to API server" → API endpoint unreachable
```

```
# - "certificate signed by unknown authority" → cluster CA mismatch
```

STEP 4: COMMON ROOT CAUSES

A. IAM Issue – Node IAM role not in aws-auth ConfigMap

```
kubectl describe configmap aws-auth -n kube-system
```

```
# Should contain the node role ARN
```

```
# Fix: Terraform aws_eks_cluster_auth + aws_auth module
```

B. VPC Endpoint missing (private cluster)

```
# Private EKS requires these endpoints:
```

```
# ecr.api, ecr.dkr, s3, sts, ec2, elasticloadbalancing
```

```
# If missing → nodes can't pull ECR images or call AWS APIs
```

```
aws ec2 describe-vpc-endpoints --filters Name=vpc-id,Values=vpc-xxx
```

C. Security Group issue

```
# Node SG must allow outbound to control plane SG on port 443
```

```
# Control plane SG must allow inbound from node SG on port 443, 10250
```

D. Subnet issue – no available IP addresses

```
aws ec2 describe-subnets \
```

```
  --subnet-ids subnet-xxx \
```

```
  --query "Subnets[0].AvailableIpAddressCount"
```

```
# If 0 → subnet exhausted → add new subnet or prefix delegation
```

E. AMI mismatch – wrong EKS-optimized AMI for K8s version

```
# Check: AMI used matches cluster K8s version
```

```
aws ssm get-parameter \
  --name /aws/service/eks/optimized-ami/1.30/amazon-linux-
2/recommended/image_id
```

STEP 5: RESOLUTION + PREVENTION

- └─ Fix the specific root cause
- └─ Add to Terraform: `aws_ec2_tag` for easier identification
- └─ Add to CI: validate `aws-auth` configmap after cluster creation
- └─ Add to runbook: "EKS node not joining" decision tree

3. Customer Delight Stories

The director will ask: **"Tell me about a time you went beyond the SLA to delight a customer."** Have 3 stories ready. Use STAR format. End with the emotional/business impact.

Story 1: Proactive Anomaly — Saving a Customer Before They Know

Situation: Was reviewing CloudWatch dashboards on a quiet Friday afternoon. Noticed a gradual memory trend on a manufacturing client's data ingestion service — memory growing 5MB per request over 8 hours. No alert had fired (threshold was 85%, they were at 67%).

Task: No ticket, no escalation, no SLA breach yet. But the math showed OOM in ~14 hours — Saturday midnight.

Action:

1. Created a Jira ticket myself, tagged it P2.
2. Dug into the code — found a Pandas DataFrame not being explicitly released after processing.
3. Fixed it in a 4-line PR, pushed through fast-track review (got second pair of eyes via Slack).
4. Deployed at 7 PM Friday using canary rollout — 100% stable by 8 PM.
5. Sent a customer-facing note: "We proactively identified and resolved a potential service degradation. No customer impact. Here's what we fixed."

Result: Customer's CTO replied: "We didn't even know about this. The fact that you caught it before we noticed is the kind of partnership we were hoping for." Renewed their contract 2 months early. Account expanded by 40%.

Lesson for Mareana: Manufacturing clients operate 24/7. A Saturday night outage could halt a production line. Being proactive — not reactive — is the difference between a vendor and a strategic partner.

Story 2: Reducing Time-to-Value for a New Customer Onboarding

Situation: A new enterprise client signed a contract. Standard onboarding was 6 weeks (environment provisioning, access setup, data pipeline configuration, testing). The client needed to demonstrate ROI to their board in 4 weeks.

Task: Cut onboarding from 6 weeks to 4 weeks without cutting corners on security.

Action:

1. Mapped the 6-week timeline to find where time was actually spent: 40% was waiting (IAM approvals, VPN setup, DNS propagation, manual email chains).
2. Built a **Customer Onboarding Terraform module** that provisioned their isolated namespace, IAM roles, network policies, S3 bucket, and ECR access in a single `terraform apply` (20 minutes vs. 3 days manual).
3. Created a self-service portal: customer uploaded their data schema → system auto-generated their ingestion pipeline YAML.
4. Ran security review and pen test in parallel (not sequentially).
5. Delivered in 3.5 weeks.

Result: Client presented their first ML-derived insights to their board with half a week to spare. They brought in two partner companies as referrals within 60 days.

What I do differently now: Every new onboarding complexity becomes a Terraform module. We've reduced onboarding from 6 weeks to 2 weeks standard.

Story 3: 3 AM Production Incident — Customer Was Watching

Situation: Production EKS cluster hit a cascading failure at 3 AM — a bad Karpenter configuration caused node thrashing (nodes launching and terminating every 90 seconds). 70% of pods were unavailable. The client's manufacturing plant shift change at 4 AM depended on the system.

Task: Resolve before 4 AM. Client CTO was online watching the status page.

Action:

1. On-call acknowledged in 3 minutes. Identified Karpenter thrashing via logs in 8 minutes.
2. Immediate containment: suspended Karpenter consolidation, scaled manually to 10 On-Demand nodes.
3. Pods recovered by 3:37 AM — 57 minutes before shift change.
4. Sent real-time updates in the client's Slack channel every 10 minutes throughout.
5. Stayed on the call with the client's ops team until 4:30 AM to ensure shift handover was clean.
6. Sent a detailed post-mortem at 9 AM with root cause, fix, and 3 preventive measures.

Result: The client CTO wrote to our VP: "Your team saved our shift changeover. The communication throughout the incident was exceptional — we always knew what was happening. This is why we chose you."

What changed permanently: Added Karpenter consolidation policy tests to our staging deployment pipeline. Wrote a runbook for Karpenter thrashing. Added a Prometheus alert: `karpenter_nodes_terminating > 3 in 5min`.

4. Trade-off & Decision-Making Questions

Q: "SageMaker vs. Self-hosted ML inference on EKS — how do you decide?"

My decision framework:

Factor	SageMaker	EKS (Self-managed)
Setup time	Hours	Days-weeks
Operational overhead	Low (managed)	High (you manage everything)
Cost (light traffic)	Higher (\$0.048/hr ml.g4dn.xlarge)	Lower (Spot GPU, shared nodes)
Cost (heavy traffic)	Lower (multi-model endpoints)	Variable
Customization	Limited (SDK-constrained)	Full (any framework, any config)
GPU utilization	Good (managed batching)	Excellent (Triton, vLLM)
Vendor lock-in	Yes (SageMaker-specific)	No (portable ONNX/TorchServe)
Multi-cloud	No	Yes (container is portable)
LLM serving	Limited	Excellent (vLLM, TGI)

My recommendation for Mareana:

"For standard ML models (tabular, computer vision): SageMaker — faster to production, managed auto-scaling, built-in monitoring. For LLMs and custom inference requiring high GPU utilization: self-hosted on EKS with NVIDIA Triton. For cost optimization at scale: multi-model SageMaker endpoints for smaller models, EKS GPU pool for large models. Don't pick one — use both for the right use cases."

Q: "ArgoCD vs. FluxCD — which GitOps tool would you pick for Mareana and why?"

Aspect	ArgoCD	FluxCD
UI	Rich UI (great for operations teams)	Minimal UI (CLI-first)
Multi-cluster	ArgoCD ApplicationSets	Flux multi-cluster (more complex)
Helm support	Excellent (native Helm)	Excellent (HelmRelease CRD)
RBAC	Fine-grained Projects + RBAC	RBAC via Kubernetes native
Notifications	ArgoCD Notifications controller	Notification controller
OCI repos	Yes (Helm OCI)	Yes
Learning curve	Moderate	Steep

My pick for Mareana: ArgoCD.

"Mareana is a product company with multiple teams. ArgoCD's UI gives platform engineers, product teams, and site reliability engineers visibility into deployment state without needing kubectl access. The Application-of-Applications pattern lets me manage 30+ services cleanly. The multi-tenant project model maps well to Mareana's team structure. FluxCD is excellent but better for pure GitOps purists — ArgoCD balances usability with power."

Q: "A developer wants to deploy directly to production via kubectl. How do you handle it?"

The wrong answer: "I deny it and lock down access."

The right answer (Director-level):

"First I ask: *Why are they asking?* Usually it's because the deployment pipeline is too slow or too complex. If it takes 45 minutes to deploy a one-line config change through CI/CD, people will find workarounds."

Short-term: Grant temporary elevated access with conditions:

- Must pair with another engineer
- Must document the change in Jira
- Audit: CloudTrail + kubectl audit logs capture everything

Medium-term: Fix the root cause:

- Fast-track pipeline for config changes (< 5 minutes, no test suite)
- Emergency deployment runbook with required approvals
- Self-service dashboard for common operations (scale up, restart, rollback)

Never: Deny without offering an alternative. If engineers bypass the system, the system is the problem, not the engineer.

Principle I follow: 'Make the right path the easy path. Make the wrong path the hard path.'

Q: "We're over budget by 30% on AWS. Engineering leadership wants you to cut costs in 2 weeks. What's your plan?"

Immediate actions (Day 1–3, no-risk):

1. Unused resources (immediate impact):
 - └─ Unattached EBS volumes → delete (run `aws-resource-cleanup` Lambda)
 - └─ Idle Elastic IPs → release (\$3.65/IP/month × many = significant)
 - └─ NAT Gateway in dev → replace with VPC endpoints where possible
 - └─ Non-prod EKS clusters → auto-shutdown 7 PM–9 AM (60% saving)
2. Right-sizing (Day 2–3):
 - └─ AWS Compute Optimizer → implement top 5 recommendations
 - └─ RDS: dev/staging → smaller instance class (`db.t3.medium`)
 - └─ Review CloudWatch: EC2 instances < 5% CPU → downsize

Week 1–2 actions (medium-risk, test first):

3. Spot Instances via Karpenter:
 - └─ Enable spot for non-prod workloads
 - └─ 60–70% savings on EC2 costs
 - └─ Risk: spot interruptions → test with `PodDisruptionBudgets`
4. S3 lifecycle policies:

- └ Old logs (>30 days) → Glacier Instant Retrieval
- └ Old backups (>90 days) → Glacier Deep Archive
- └ Estimated: 40-50% S3 cost reduction

5. Data Transfer:

- └ Add VPC Gateway Endpoints for S3 + DynamoDB (free, instant)
- └ Saves NAT Gateway data charges (\$0.045/GB × high-volume traffic)

Communicate transparently:

"I can cut 20-25% in 2 weeks with low risk.
Cutting 30% requires also touching production (Spot for some workloads), which needs 2 more weeks for proper testing.
I recommend we target 20% in 2 weeks and 30% in 4 weeks safely.
Rushing the last 10% risks a production incident that costs more than it saves."

5. Platform Strategy & Roadmap Thinking

Q: "How would you build a Platform Engineering practice at Mareana from scratch?"

Answer:

"Platform Engineering is about creating an Internal Developer Platform (IDP) that gives product teams developer self-service without sacrificing security or reliability. Here's my 12-month roadmap:"

Quarter 1 — Foundation

Goal: Developers can deploy to EKS without needing help from platform team

Deliverables:

- └ "Golden path" CI/CD template (GitHub Actions → ECR → ArgoCD)
 - Any team can use it, no platform ticket needed
- └ Service catalog: Terraform modules for common patterns

- | (microservice + RDS, microservice + SQS, ML service + GPU)
- └ Developer documentation portal (internal Backstage or Confluence)
- └ Observability out-of-the-box: Prometheus + Grafana dashboard per service (launch a service → get monitoring automatically)

Quarter 2 — Self-Service

Goal: Platform team is not a bottleneck

Deliverables:

- └ Backstage developer portal:
 - └ Software catalog (every service, its owner, its SLAs)
 - └ Self-service: "Create new microservice" → GitHub repo + EKS namespace created
- └ Docs hub: runbooks, ADRs, incident history
- └ Cost dashboard: per-team spend
- └ SLO framework: every service defines SLIs + SLOs
 - Prometheus rules auto-generated from SLO definition
- └ On-call rotation system (PagerDuty teams + escalation paths)

Quarter 3 — Reliability

Goal: 99.9% uptime, 30-minute incident resolution

Deliverables:

- └ Chaos Engineering: quarterly Game Days with AWS FIS
- └ DR testing: bi-annual region failover test
- └ Runbook automation: common incidents → automated resolution (pod OOMKilled → auto-restart + alert, not just alert)
- └ SLO review: monthly cross-team SLO meeting

Quarter 4 — Innovation

Goal: Platform enables Mareana's competitive differentiation

Deliverables:

- └ Edge computing: AWS Greengrass for factory-floor AI inference
 - └ Multi-cloud DR: secondary cloud (Azure/GCP) for extreme resilience
 - └ AI-assisted ops: LLM-powered incident triage (alert fires → LLM suggests likely cause + link to past incidents)
 - └ Platform roadmap published quarterly: show teams what's coming
-

Q: "What's the biggest platform mistake you see companies make?"

Answer:

"Building a platform for engineers, not with engineers. The platform team goes dark for 3 months, builds something beautiful based on their own assumptions, launches it — and no one uses it because it doesn't match how product teams actually work.

The fix: Treat the Internal Developer Platform like a product. It has customers (your engineers), a roadmap, a feedback loop, and adoption metrics. Run monthly user interviews. Measure 'time from idea to production deployment' as your north star metric. If that number isn't decreasing, your platform isn't working.

At Mareana, I'd start with a 2-week discovery sprint: shadow 3 different engineering teams, document their pain points, then build the platform to solve their top 3 pains. Not the top 3 things I think are cool."

6. Failure & Learning Questions

Q: "Tell me about the biggest production incident you caused or that happened on your watch."

Answer (be honest — directors respect vulnerability):

"During an EKS upgrade from 1.27 to 1.28, I ran the cluster upgrade on a Tuesday afternoon rather than Sunday morning (our maintenance window). I rationalized it because I'd done this upgrade successfully on staging. What I didn't account for:

The VPC CNI add-on upgrade required a brief rolling restart of all nodes. During that window, 3 pods with `maxUnavailable: 0` were stuck pending because the new CNI version had a 90-second initialization delay. Those were our highest-traffic APIs. We had degraded performance (not outage) for 22 minutes — 15% elevated error rate.

What I did: Declared a P2, communicated to stakeholders, rolled through the CNI update faster by pre-scaling to double capacity before the upgrade.

What I learned permanently:

1. EKS upgrades are *always* done during maintenance windows. No exceptions.
2. CNI upgrades need a specific runbook step: pre-scale nodes by 50% before upgrade.
3. I now have a 15-item pre-upgrade checklist. Item 1: 'Is today our maintenance window? Y/N. If N, stop.'

The blameless post-mortem was uncomfortable because I was the decision-maker. But the outcome was a much stronger upgrade process that my whole team now follows."

Q: "Tell me about a technology decision you made that you later regretted."

Answer:

"Early in my career, I recommended Helm for managing all our Kubernetes configuration across environments — dev, staging, prod. It worked fine for simple services. But as we grew to 40+ microservices, our Helm values files became massive, we had charts-within-charts, and a values override in the wrong place caused a production difference that wasn't visible in staging.

The real problem: Helm is a templating system, not a state management system. It doesn't tell you what's currently deployed — only what you intended to deploy. When drift happened, we had no easy way to detect it.

I moved us to ArgoCD + Kustomize overlays. ArgoCD continuously compares desired state (Git) with actual state (cluster) and tells you exactly what drifted. The migration took 3 weeks and was painful. But our deployment confidence went up significantly.

What I'd do differently: Use the right tool for the right job. Helm for packaging and distribution. Kustomize + ArgoCD for environment-specific configuration and state management. Never use Helm alone for multi-environment CD."

7. Leadership, Influence & Cross-Functional

Q: "How do you get a security team to approve a Kubernetes architecture when they don't understand containers?"

Answer:

"This is a real challenge. Security teams are trained on perimeter security models — firewalls, network segmentation, VLANs. Kubernetes can look like chaos to them.

My approach:

Step 1: Speak their language, not mine. Instead of: 'We use NetworkPolicies for pod isolation.' Say: 'Each service can only talk to the services it needs to — enforced at the network layer, auditable, immutable.'

Step 2: Map K8s concepts to familiar equivalents:

- Namespace = VLAN
- NetworkPolicy = Firewall rule
- RBAC = Active Directory group policies
- IRSA = Service accounts with scoped permissions
- OPA/Gatekeeper = Change management guardrails

Step 3: Run a joint threat model session. Not 'here's what we built' — but 'help me find the gaps.' Security teams become allies when they're in the design process, not gatekeepers reviewing a finished product.

Step 4: Give them the audit trail they need. CloudTrail for every API call. Kubernetes audit logs. Falco alerts for anomalies. Security teams need to be able to answer: 'Who did what, when, to what resource.'

Result: When security is part of the design, approvals come faster. I've reduced security review cycles from 6 weeks to 2 weeks using this approach."

Q: "How do you handle a product manager who wants to skip the staging environment to ship faster?"

Answer:

"I understand the pressure — every day in staging is a day a customer isn't getting value. So I don't just say 'no' to skipping staging. I ask: what's the actual risk they're trying to avoid?"

Usually it's one of three things:

1. Staging is too slow (pipeline takes 2 hours)
2. Staging is unreliable (doesn't mirror production)
3. They've been waiting for 2 days and something changed in production anyway

For each:

1. I invest in making staging faster — 20-minute fast-track pipeline for low-risk changes.
2. I make staging mirror production — same instance sizes, same data shape (anonymized), same configuration. 'Prod-parity staging' removes the 'but it worked in staging' excuse.
3. I introduce feature flags — ship the code to production, hidden behind a flag. Enable for 1% → 10% → 100%. Production is now the test environment with guardrails.

The conversation I have with PMs: 'I want to ship as fast as you do. Let's make the pipeline fast enough that skipping staging is never the right answer.'

I never win an argument by being the 'safety person.' I win by making safety fast."

Q: "How do you influence technical decisions when you don't have direct authority — e.g., a product team that reports to a different VP?"

Answer:

"Influence without authority is mostly about credibility and relationship, not hierarchy.

What works for me:

1. **Fix their problem first, influence later.** If I help a product team solve a hard problem, they trust my architectural recommendations in the future.
2. **Write the ADR (Architecture Decision Record), not the mandate.** ADRs document options, trade-offs, and recommendations. Engineers can engage with reasoning. Mandates create resistance.
3. **Show don't tell.** Instead of 'you should use Karpenter,' I say 'here's the Karpenter cost dashboard for our other teams — they saved 35% last quarter.' Numbers move people.
4. **Make it easy to do the right thing.** If my recommendation requires 3 days of work, it won't get adopted. If I provide a Terraform module that can be used in 20 minutes, it will. The path of least resistance should be the secure, scalable, correct path.
5. **Accept losing some battles.** If a team makes a decision I disagree with, I document my concerns in the ADR, agree to monitor the outcome, and revisit in 3 months. Credibility comes from being right over time, not from winning every argument."

8. Mareana Domain Deep-Dives

Q: "How would you architect the data pipeline for a manufacturing plant producing 10,000 sensor readings per second?"

Answer:

MANUFACTURING IOT PIPELINE – 10,000 events/second:

EDGE LAYER (Factory Floor):

- Sensors → PLCs/SCADA → AWS IoT Greengrass (factory gateway)
- Greengrass local rules:
 - Filter noise (only send out-of-threshold readings)
 - Local ML inference (anomaly detection at edge – <1ms latency)
 - Buffer during connectivity loss (store-and-forward)
- Protocol: MQTT (lightweight for sensors), HTTPS for bulk
- Bandwidth optimization: send delta only (not full reading if unchanged)

INGESTION LAYER (AWS):

- AWS IoT Core → IoT Rule → Kinesis Data Streams
 - Shards: $10,000 \text{ events/sec} \times \sim 200 \text{ bytes} = 2 \text{ MB/sec} = 2 \text{ shards}$ (buffer for spikes: 10 shards)
 - Retention: 7 days (replay capability)
 - Enhanced fan-out: separate consumers don't compete for throughput
- Kinesis Firehose → S3 (raw data, Parquet, partitioned by plant/date/hour)
- IoT SiteWise: structured asset model for equipment hierarchies

REAL-TIME PROCESSING (< 1 second):

- Kinesis Analytics (Apache Flink)
 - Sliding window (30s): detect spike in temperature sensor
 - CEP (Complex Event Processing): 3 consecutive anomalies = alert
 - Output: DynamoDB (real-time dashboard) + SNS (alerts)
- Lambda: threshold breach → SNS → PagerDuty → maintenance engineer

BATCH PROCESSING (hourly/daily):

- AWS Glue Spark job: S3 Bronze → Silver (clean, join, enrich)
- S3 Silver → Redshift Serverless (analytics, BI reporting)
- SageMaker: retrain predictive maintenance model on new 24h data

STORAGE STRATEGY:

- DynamoDB: last 5 minutes per sensor (real-time dashboard)
- Timestream: last 1 year (time-series queries, fast aggregation)
- S3 Parquet: 7+ years (historical analysis, ML training)
- Redshift: aggregated metrics (daily/weekly reports)

SCALE MATH:

$10,000 \text{ events/sec} \times 200 \text{ bytes} = 2 \text{ MB/sec ingestion}$
 $10,000 \times 86,400 \text{ seconds} = 864 \text{M events/day}$
 $864 \text{M} \times 200 \text{ bytes} = \sim 172 \text{ GB/day raw data} \rightarrow \text{compressed Parquet} \approx 17 \text{ GB/day}$
Cost: Kinesis $\sim \$7/\text{day}$, S3 $\sim \$0.40/\text{day}$, Timestream $\sim \$50/\text{day}$

Q: "Mareana's AI model predicts equipment failure. How do you ensure that model keeps working accurately in production 12 months from now?"

Answer:

ML MODEL RELIABILITY IN PRODUCTION – PREDICTIVE MAINTENANCE:

CHALLENGE: Manufacturing data drifts constantly:

- └ Seasonal variation (temperature affects sensor baselines)
- └ Equipment aging (baseline readings shift over months)
- └ Maintenance events (sensor recalibration changes readings)
- └ New equipment added (model never saw this sensor profile)

SOLUTION: ML MODEL HEALTH MONITORING SYSTEM:

1. BASELINE CAPTURE (at deployment):

- └ Record: distribution of all input features (mean, std, percentiles)
- └ Record: model output distribution (failure probability distribution)
- └ SageMaker Model Monitor: auto-creates baseline statistics
- └ Store in S3 + DynamoDB: feature baseline per plant per equipment type

2. CONTINUOUS DATA DRIFT DETECTION:

- └ SageMaker Data Quality Monitor (hourly):
 - └ Compare: current input features vs. baseline
 - └ Statistical tests: KS test (continuous), chi-squared (categorical)
 - └ Alert if: any feature drift score > 0.1
 - └ Dashboard: feature drift heatmap (all sensors $\times$ drift score)
- └ Custom metrics (domain-specific):
 - └ Sensor reading variance vs. historical (Timestream query)
 - └ Operating hours since last maintenance (affects baseline)
- └ Alert: Prometheus $\rightarrow$ AlertManager $\rightarrow$ Slack (platform team + data science team)

3. MODEL PERFORMANCE MONITORING:

- └ Ground truth collection:
 - └ When equipment fails $\rightarrow$ label the prediction that preceded it
 - └ Automation: CMMS (maintenance system) webhook $\rightarrow$ Lambda $\rightarrow$ label store
- └ Weekly batch: compute actual precision, recall, F1 on last 7 days
- └ SageMaker Model Quality Monitor:
 - └ Compare: model quality metrics vs. baseline (AUC, F1)
 - └ Alert if: F1 drops $> 5\%$ below baseline
- └ Business metric monitoring:
 - └ Unplanned downtime events (should decrease)
 - └ False alarm rate (maintenance dispatched but no failure found)

4. AUTOMATED RETRAINING TRIGGER:

Trigger if ANY of:

- └ Data drift score > 0.15 for 3 consecutive days
- └ Model F1 $< (\text{baseline_F1} \times 0.9)$ for 7 days
- └ New equipment type added to plant
- └ Scheduled: monthly full retraining regardless (prevent silent decay)

5. RETRAINING PIPELINE:

EventBridge trigger $\rightarrow$ SageMaker Pipeline:

- └ Step 1: Pull last 6 months of labeled data
- └ Step 2: Data quality validation (Great Expectations)
- └ Step 3: Train new model (same hyperparameters $\rightarrow$ compare)
- └ Step 4: Evaluate: new vs. current model on holdout set
- └ Step 5: If new model $>$ current by 2%: promote to staging

- Step 6: Shadow test (both models run in parallel, only current serves)
- Step 7: If shadow test confirms improvement: canary → 100%
- Step 8: If regression: auto-reject, keep current, alert data science team

6. CUSTOMER COMMUNICATION:

- Monthly model health report per customer
 - "Your model accuracy: 94.2% (↑ 1.1% from last month)"
- Proactive: "We retrained your model with your latest 6 months of data"

9. Director's Rapid-Fire Round

Directors often do a rapid-fire at the end — quick questions expecting concise, confident answers. Practice saying these without hesitation.

Question	Strong Answer
What's your first week at Mareana look like?	"Listen, observe, no changes. Shadow on-call. Meet every team. Map the existing architecture. Identify the top 3 pain points before proposing any solutions."
ECS vs EKS — pick one for Mareana and defend it.	"EKS. Mareana's ML workloads need GPU scheduling, complex networking, and community ecosystem. ECS is fine for simple services but can't serve the complexity ahead."
What keeps you up at night as a platform engineer?	"Silent failures — the things monitoring doesn't catch. A misconfigured resource quota that's not blocking yet. A security group rule that's too permissive but not exploited yet."
How do you know when you've won the trust of engineering teams?	"When they come to you with problems before they become incidents. And when they push back on my ideas because they trust I'll listen."
What's one thing you'd change about how most companies do DevOps?	"Measuring deployment frequency and lead time but not developer happiness. Toil that doesn't cause incidents never gets fixed. I track 'hours spent on manual work per engineer per sprint' as a real metric."

Question	Strong Answer
What's your biggest weakness as an architect?	"I sometimes over-engineer for scale that doesn't exist yet. I've learned to ask 'what's the cost of this decision if we're wrong in 12 months?' before adding complexity."
What AWS service do you think is underused?	"AWS EventBridge Pipes. It's a point-to-point integration service that eliminates a lot of Lambda glue code. Most teams don't know it exists."
How do you stay current with AWS announcements?	"AWS weekly newsletter, re:Invent recordings, AWS blog, and running a monthly team session where we review what's changed and what we should adopt."
If you had to deprecate a tool your team loves, how do you do it?	"Give 90 days notice. Provide a migration guide. Offer help with migration. Never remove a tool without its replacement being ready and proven."
Rate your Kubernetes knowledge 1-10 and justify.	"8. I've run production clusters at scale, done upgrades, handled security hardening, built autoscaling with Karpenter. The gap to 10 is networking internals (eBPF, CNI deep code) and being current on the K8s 1.30+ feature set which I'm actively closing."

10. Questions YOU Should Ask the Director

Asking sharp questions signals strategic thinking. Ask 3–4 from this list.

About the Platform Today:

1. *"What's the current biggest gap between what the platform can do today and what the product team needs it to do in 12 months?"*
2. *"What's the on-call experience like today — how often are engineers paged, and what percentage of pages are actionable vs. noisy?"*

3. *"What's the most painful deployment failure you've had in the last 6 months, and what did it teach you?"*

About the Role: 4. *"What does success look like for this role at 30, 60, and 90 days?"*

5. *"Is this primarily a build role (greenfield) or an improvement role (existing platform)? What's the ratio?"*
6. *"Who are the internal 'customers' of the platform team — product engineering, data science, both?"*

About Mareana's Scale & Direction: 7. *"As Mareana moves into more manufacturing verticals, how does the platform need to evolve — more edge computing? More multi-tenancy? More data sovereignty requirements?"*

8. *"How does the platform team interact with the ML/AI product teams? Is there embedded collaboration or more of a ticket-based model today?"*

About the Team: 9. *"What does the current platform team look like, and what skill gaps are you hoping this hire addresses?"*

10. *"What's the engineering culture around 'we broke production' — is it blameless or is there still fear of failure?"*
-

Final Prep Checklist Before Your F2F

48 hours before:

- ☐ Re-read L1 doc — refresh core technical answers
- ☐ Practice the 3 Customer Delight stories out loud (timer: 3 min each)
- ☐ Practice Debug Scenario 1 (ML latency) out loud — narrate every step
- ☐ Review Mareana's website — any new product announcements, case studies, blog posts?
- ☐ Review the Director's LinkedIn — understand their background, what they care about

Day of interview:

- ☐ Bring a notebook — take notes when director explains problems (shows listening)

- ☐ Have your 5 strongest stories mentally labeled: Architecture, Debug, Customer Delight, Failure, Leadership
 - ☐ Ask your prepared questions at the end — not "any questions?" energy, but genuinely curious
 - ☐ Close strong: "I'm excited about this role. The manufacturing + AI domain with the platform engineering challenge at Mareana's scale is exactly the kind of work I want to be doing."
-

The One Thing to Remember

Directors don't hire people who know the right answers. Directors hire people who think the right way.

Show them:

- How you **structure ambiguity** into a clear problem
 - How you **stay calm** under production pressure
 - How you **earn trust** from engineers and customers
 - How you **think 12 months ahead** while executing today
-

Prepared for: Pushparaj Naik | Role: Cloud & DevOps Architect — Mareana | Round: Final F2F (Director) Complements:

Marena_Cloud_DevOps_Architect_Interview_Preparation.md (L1) Prepared: June 2026